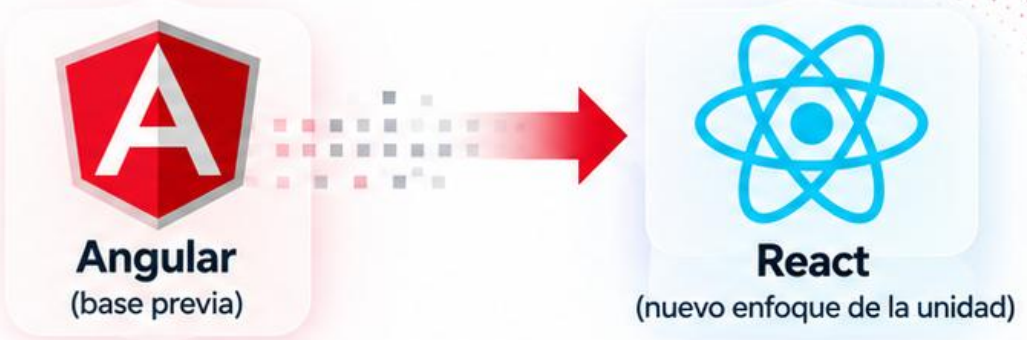
Universidad Tecnológica del Perú

Introducción a React

De Angular a una nueva forma de construir interfaces



- **Curso:** JavaScript Avanzado.
- **Unidad 3:** React.
- **Semana 11.**
- **Docente:** *Mg. Ivan Eduardo Anchante Reyes.*







¿Qué conocimientos nos llevamos de Angular?

El framework cambia; varias ideas permanecen

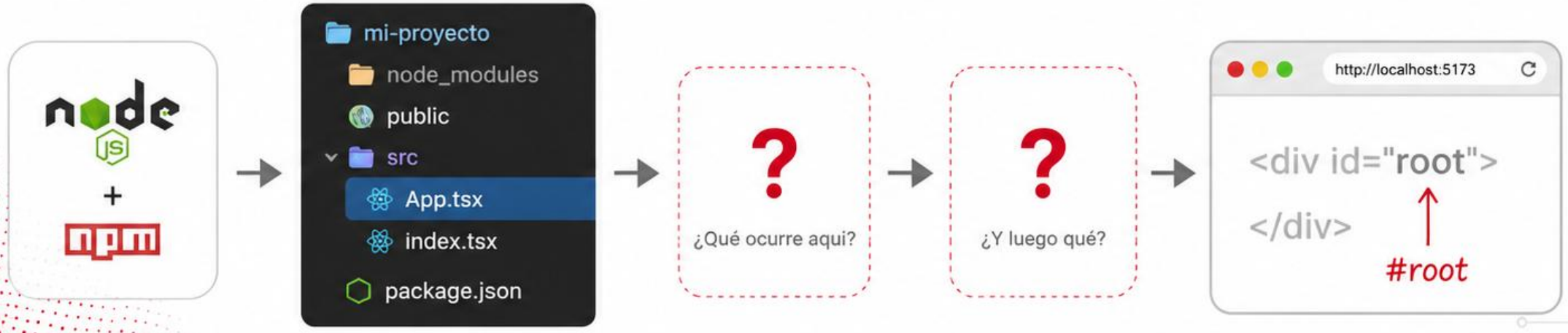


Pregunta: ¿qué idea crees que también encontraremos en React?

Diagnóstico: ¿qué hace falta para ver una interfaz?

Ordena mentalmente el flujo

Tenemos Node.js, un proyecto, un archivo **App.tsx y un navegador.**
¿Qué debe ocurrir entre el código y la interfaz visible?
¿Quién inicia el renderizado? ¿Dónde se monta React?



¿Por qué aprender React después de Angular?

No se trata de memorizar otra sintaxis



Pregunta: ¿qué valor tiene poder **reconocer los mismos principios** de interfaz en más de una tecnología?

Transferencia esperada: proyecto, componentes, datos, estilos y construcción incremental de UI.

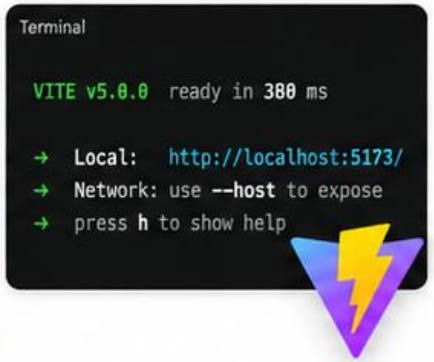
Aprender **principios** para **aplicarlos** en cualquier tecnología.

Logro de la sesión

Qué debes demostrar al finalizar

Al finalizar la sesión, el estudiante crea y ejecuta un proyecto React con Vite, interpreta JSX y TSX, integra Bootstrap y construye una interfaz web básica con datos tipados.

1 Vite



2 React / TSX



3 Bootstrap



4 Interfaz web



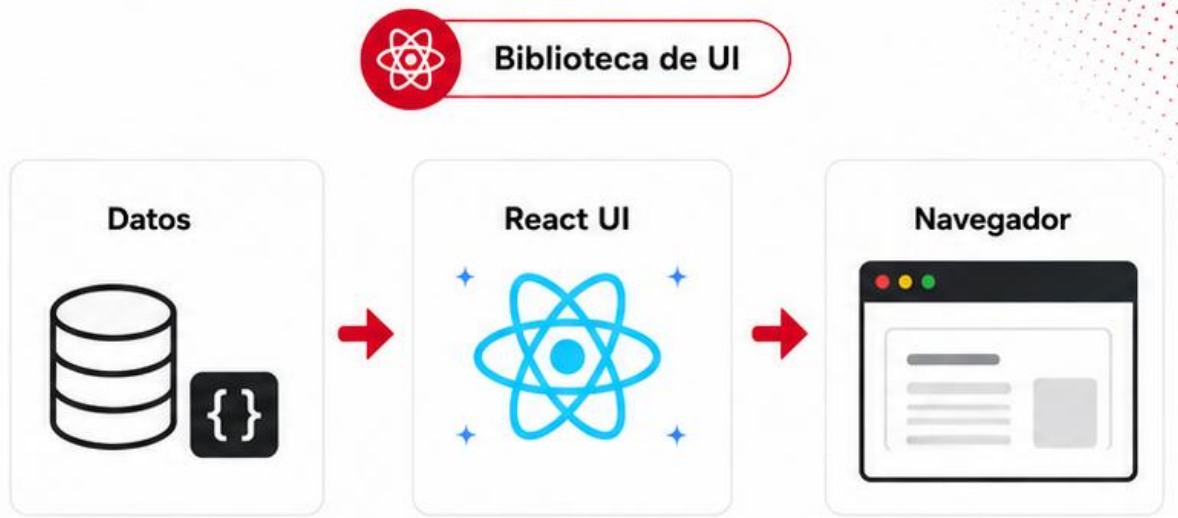
¿Qué es React?

Una biblioteca para construir interfaces de usuario

React permite describir interfaces mediante componentes y datos. En esta semana solo necesitamos comprender el punto de entrada, *App*, JSX/TSX y la representación de información. Navegación, formularios y componentes más elaborados quedan para la siguiente semana.

Alcance de esta semana

Nos enfocaremos en el punto de entrada (*App*), JSX/TSX y la representación de datos en la UI.



App.tsx

```tsx
type ContadorProps = { inicial: number };

export default function App() {
  const [contador, setContador] = useState(0);
  return (
    <div className="container">
      <h1>Hola React 🚀</h1>
      <p>Contador: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>
        Incrementar
      </button>
    </div>
  );
}
```



Modelo mental mínimo

React monta una aplicación dentro de una página

`index.html` contiene el nodo `#root`. `main.tsx` crea el root de React y renderiza `<App />`. `App.tsx` devuelve la interfaz que vemos.

1 index.html
La página define el contenedor

index.html

```html
<!DOCTYPE html>
<html lang="es">
  <body>
    <div id="root"></div>
  </body>
</html>
```

nodo **#root**

2 main.tsx
Crea el root e inicia React

TSX

```tsx
import { createRoot }
from 'react-dom/client'
import App from './App'

// Crea el root en #root
const root = createRoot(
  document.getElementById('root')!
)
root.render(<App />)
```

3 <App />
Componente raíz

TSX

```tsx
export default function App() {
  return (
    <div className="app">
      <h1>Hola React 🚀</h1>
      <p>Contador: 1</p>
    </div>
  )
}
```

4 Pantalla
Lo que el usuario ve



Configuración actual para aprender React

Vite reemplaza el flujo antiguo de Create React App

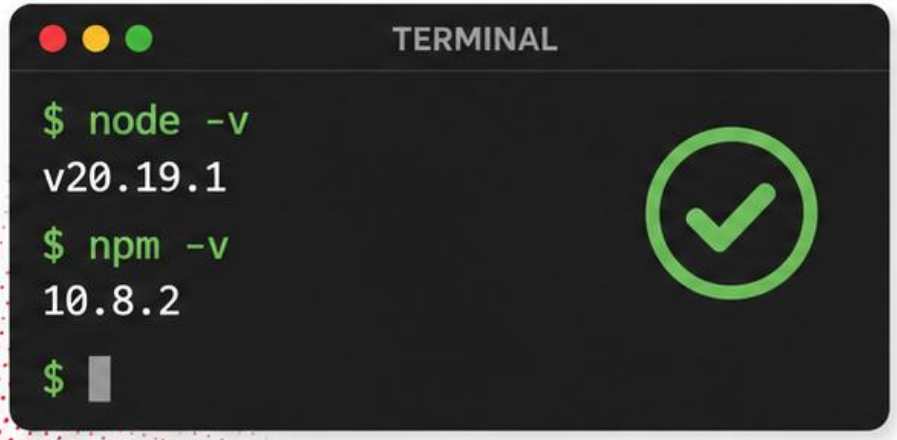
Para esta clase usaremos **Vite.**
React desaconseja **Create React App** porque está **deprecado.**
Vite permite crear rápidamente un proyecto React con TypeScript y ejecutar un servidor de desarrollo.



Antes de crear el proyecto

Comprueba Node.js y npm

En terminal: **node -v** y **npm -v**. Vite actual requiere Node.js 20.19+ o 22.12+. Si el equipo no cumple, actualiza Node antes de continuar.



Versión requerida

Node.js
20.19+ o 22.12+

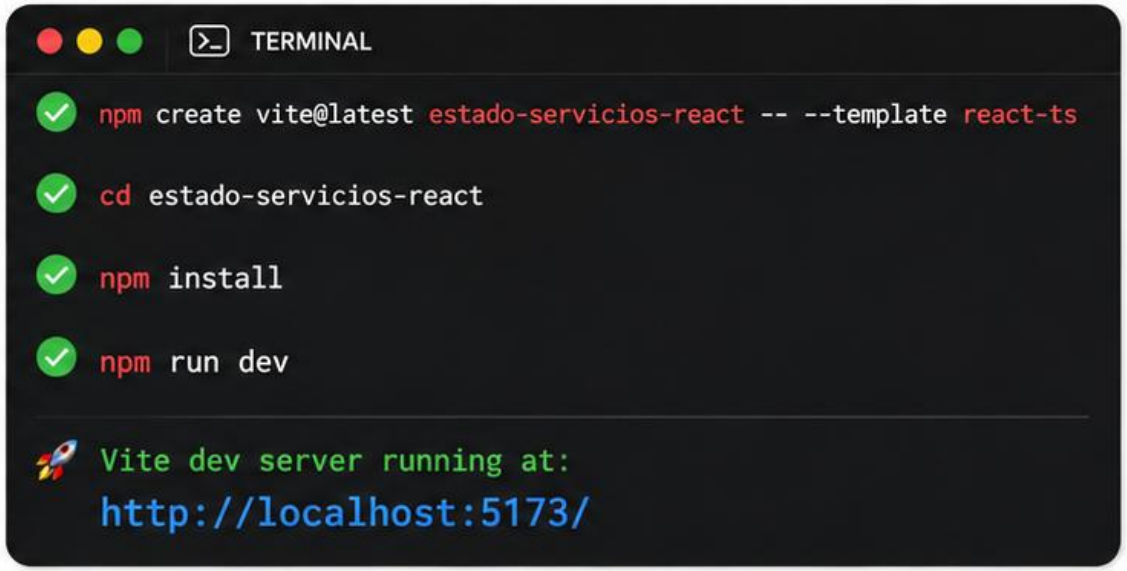
¡Todo listo!
Si tus versiones cumplen con el requisito, estás listo para crear tu proyecto con Vite.

Atención
Si la versión es menor a 20.19, actualiza Node.js antes de continuar para evitar problemas de compatibilidad.

Crear un proyecto React + TypeScript

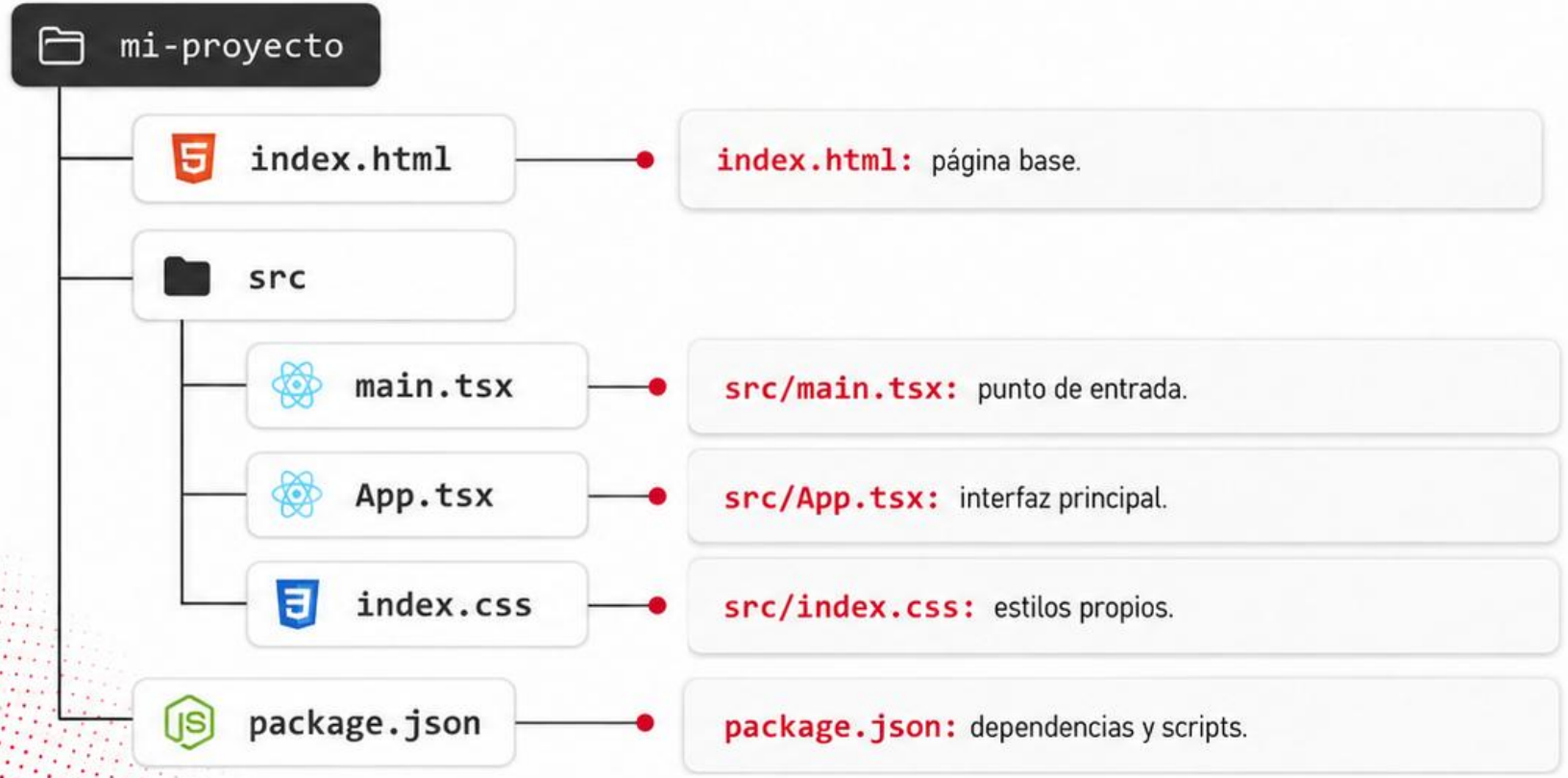
Un solo comando genera la estructura inicial

1. `npm create vite@latest estado-servicios-react -- --template react-ts`
2. `cd estado-servicios-react`
3. `npm install`
4. `npm run dev`



¿Qué archivos necesitamos reconocer?

No memorices todo el proyecto



main.tsx: el punto de entrada

Aquí React se conecta con #root

Además importamos los estilos globales y Bootstrap.

```
createRoot(document.getElementById('root')!).render(<StrictMode><App /></StrictMode>);
```

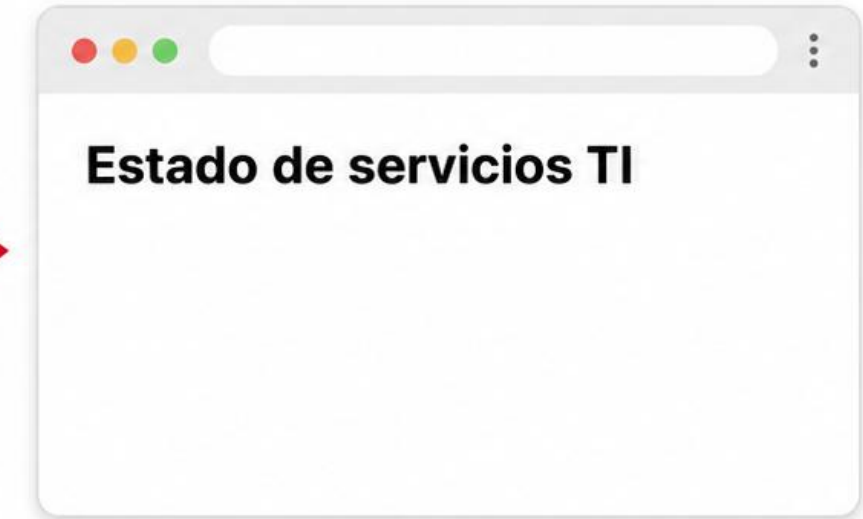


App.tsx: nuestra primera interfaz

Una función puede devolver marcado React

```
function App() {
   return (
     <main>
       <h1>Estado de servicios TI</h1>
     </main>
  );
}

export default App;
```



El nombre del componente inicia con **mayúscula**.

Universidad Tecnológica del Perú

JSX: marcado dentro de JavaScript

Se parece a HTML, pero vive dentro del código

¿Qué es JSX?

JSX permite escribir una **estructura visual** junto con la lógica JavaScript.

Un archivo puede usar extensión **.jsx** cuando trabajamos con JavaScript.

¿Cómo funciona?



Ejemplo

```
const nombre = "React";

<h1>Hola {nombre}</h1>
```

Definimos una variable en JavaScript.

Usamos JSX (parecido a HTML) e insertamos el valor de la variable con {nombre}.

Expresión insertada (se evalúa y muestra "React").

Reglas JSX que debes reconocer

Pequeñas diferencias respecto de HTML



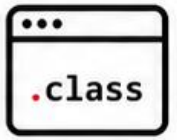
Cerrar etiquetas

`<img />`



Único padre o fragmento

`<>...</>`



className

usar `className` en lugar de `class`



Expresiones JavaScript

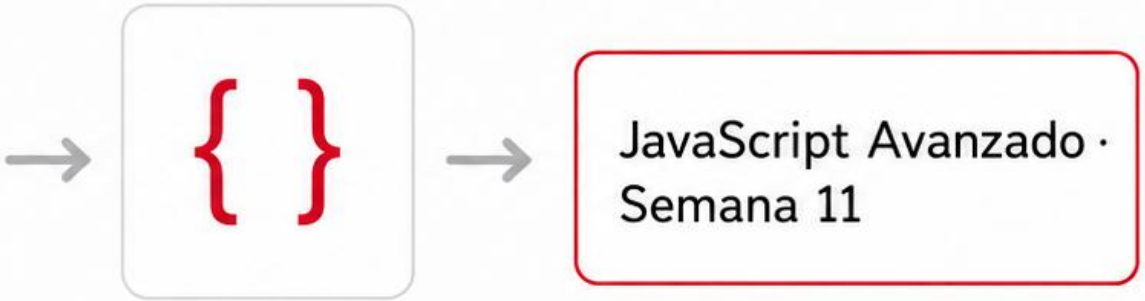
insertar expresiones entre `{ }`

Las llaves insertan expresiones

La interfaz puede mostrar datos calculados

```
const curso = "JavaScript Avanzado";
const semana = 11;

JSX: <p>{curso} · Semana {semana}</p>.
```



Dentro de { } puede ir una variable, una operación o una expresión válida.

TSX: TypeScript + JSX

El marcado es similar; agregamos tipos

Cuando un archivo TypeScript contiene JSX usa extensión **.tsx.**
Podemos declarar tipos para los datos que luego representamos en la interfaz.
React con TypeScript mantiene JSX y añade verificación de tipos.

JS **.jsx = JS + JSX**

JavaScript
Sin verificación de tipos

```jsx
function Saludo(props) {
  return (
    <div>
      <h1>Hola, {props.nombre}!</h1>
    </div>
  );
}
```

JSX para el marcado.
JavaScript maneja los datos sin tipos.

VS

TS **.tsx = TS + JSX + tipos**

TypeScript
Con verificación de tipos

```tsx
type Props = {
  nombre: string;
  edad: number;
};

function Saludo({ nombre, edad }: Props) {
  return (
    <div>
      <h1>Hola, {nombre}!</h1>
      <p>Tienes {edad} años.</p>
    </div>
  );
}
```

JSX para el marcado.
TypeScript añade verificación de tipos en tiempo de desarrollo.

Ejemplo tipado en TSX

Primero describimos la forma del dato.

```ts
interface Servicio {
  nombre: string;
  estado: 'Operativo' | 'Degradado';
  disponibilidad: number;
}
```

Objeto validado (cumple la interfaz)

```ts
{
  nombre: 'Auth Service',
  estado: 'Operativo',
  disponibilidad: 99.95
}
```

Tarjeta UI (consumo en React)



Luego un objeto o arreglo debe respetar esa estructura.

Universidad Tecnológica del Perú

Microdecisión: JSX o TSX

Identifica qué cambia realmente

A

App.jsx usa JavaScript y devuelve **<h1>**.

JSX

```jsx
export default function App() {
  return <h1>Hola mundo</h1>;
}
```

B

App.tsx declara **interface Servicio** y devuelve **JSX**.

TSX

```tsx
interface Servicio {
  id: number;
  nombre: string;
}

export default function App() {
  return <h1>Servicio activo</h1>;
}
```

C

Un archivo **.ts** intenta contener **<h1>**.

TS

```ts
export function titulo(): string {
  return <h1>Error</h1>; // ¿Permite TS?
}
```

Preguntas:

1. ¿Cuál de los tres casos usa ***TypeScript***?
2. *¿Qué extensión de archivo se necesita para escribir **JSX** dentro de **TypeScript**?*
3. *¿Qué caso debe **corregirse** y por qué?*

Instalar Bootstrap

React sigue usando clases CSS

Dentro del proyecto: `npm install bootstrap`. Luego en `main.tsx`:

```
import "bootstrap/dist/css/bootstrap.min.css";
```

Para esta semana usaremos principalmente su CSS.

1 Instalar Bootstrap → **2** Importar en `main.tsx` → **3** Tarjetas estilizadas

```
$ npm install bootstrap

added 1 package, and audited 210 packages in 2s

20 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

main.tsx

```tsx
1  import React from "react";
2  import ReactDOM from "react-dom/client";
3  import "bootstrap/dist/css/
   bootstrap.min.css";
4  import App from "./App";
5  ReactDOM.createRoot(document.getElementById
   ("root")!).render(<App />);
```



Bootstrap dentro de JSX/TSX

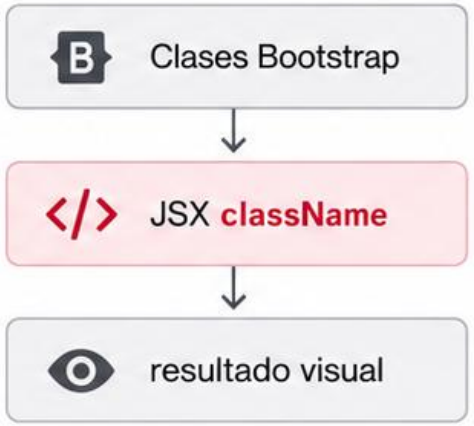
La diferencia clave es **className**

1. En JSX/TSX

```jsx
<div className="container py-4">
  ...
</div>

<span className="badge text-bg-success">
  Operativo
</span>
```

2. ¿Qué ocurre?



3. Resultado en el navegador



Las clases de Bootstrap funcionan normalmente, pero JSX usa **className**.

Demostración: Estado de servicios TI

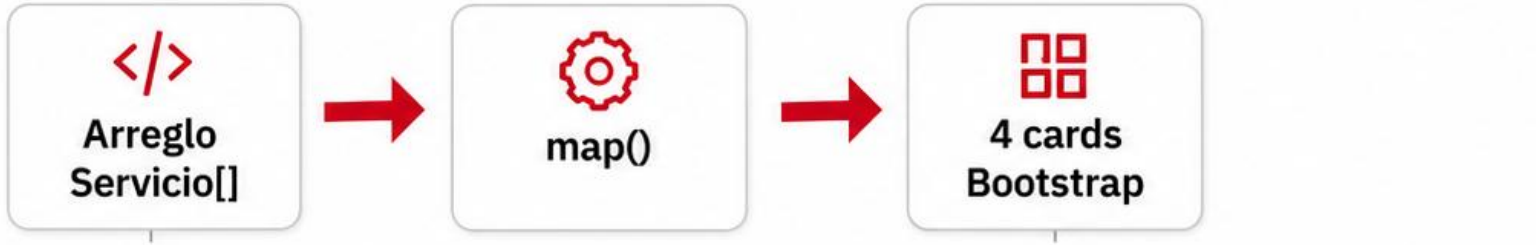
Una aplicación simple, sin navegación ni formularios

Datos: nombre, categoría, estado y disponibilidad.

Objetivo: mostrar cuatro servicios en tarjetas Bootstrap usando un arreglo tipado y map().

Caso del docente:

- Portal académico
- WiFi
- Correo
- Mesa de ayuda



TypeScript (Interfaz)

```typescript
interface Servicio {
  nombre: string;
  categoría: string;
  estado: 'operativo' | 'degradado' | 'caído';
  disponibilidad: number; // porcentaje 0–100
}
```



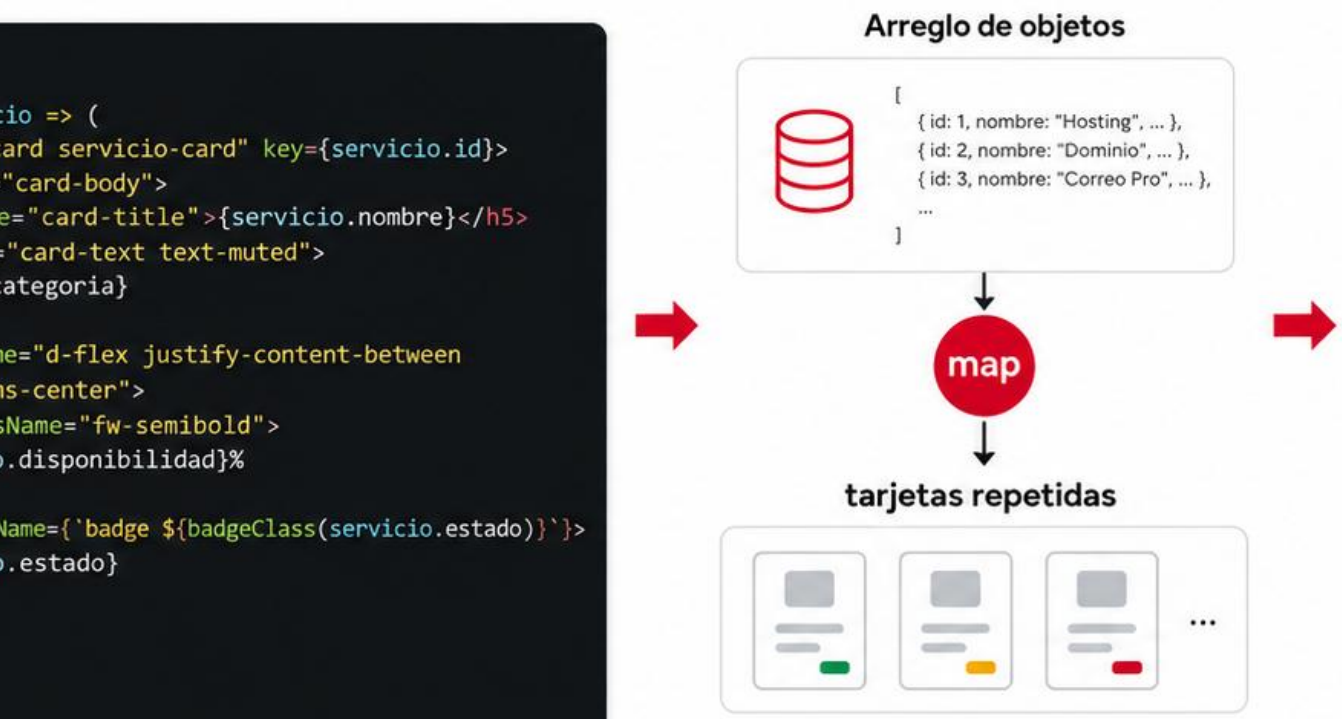
Universidad Tecnológica del Perú

Del arreglo a la interfaz

React transforma cada dato en marcado

Dentro de cada tarjeta mostramos **{servicio.nombre}**, **{servicio.categoria}**, **{servicio.disponibilidad}%** y una badge cuyo estilo depende del **estado**.

```
servicios.map(servicio => (
  <div className="card servicio-card" key={servicio.id}>
    <div className="card-body">
      <h5 className="card-title">{servicio.nombre}</h5>
      <p className="card-text text-muted">
        {servicio.categoria}
      </p>
      <div className="d-flex justify-content-between
          align-items-center">
        <span className="fw-semibold">
          {servicio.disponibilidad}%
        </span>
        <span className={`badge ${badgeClass(servicio.estado)}`}>
          {servicio.estado}
        </span>
      </div>
    </div>
  </div>
))
```



Hosting Básico — Activo
Alojamiento Web
98%

Dominio .com — Activo
Registro de Dominios
100%

Correo Pro — Mantenimiento
Correo Corporativo
76%

SSL Wildcard — Inactivo
Certificados SSL
45%

Universidad Tecnológica del Perú

Ejecutar, modificar y comprobar

El ciclo de trabajo debe ser corto

npm run dev inicia desarrollo. Cambia un nombre o disponibilidad y observa Fast Refresh. **npm run build** comprueba que el proyecto puede generar una versión de producción. **Regla:** primero logra una interfaz correcta y luego agrega complejidad.



Regla: primero logra una interfaz correcta y luego agrega complejidad.

Universidad Tecnológica del Perú

Actividad individual

Reto: Panel de laboratorios

Modalidad: **INDIVIDUAL.**

Tiempo: **60 minutos.**

Crea una aplicación **React + TypeScript** con **Vite** que muestre información de laboratorios académicos mediante **TSX** y **Bootstrap**. **No uses navegación, formularios ni Hooks.**





Requisitos del reto

Misma competencia, contexto diferente

1. crear proyecto react-ts
2. instalar/importar Bootstrap
3. definir interface Laboratorio
4. crear mínimo 4 laboratorios
5. representar con map()
6. usar className y expresiones { }
7. mostrar badge por estado
8. mostrar cantidad de laboratorios disponibles
9. diseño legible con grid Bootstrap




```
$ npx create-react-app laboratorio --template typescript
cd laboratorio
npm install bootstrap
npm install @types/react-router-dom
```

Laboratorio.tsx

```tsx
interface Laboratorio {
 id: number;
 nombre: string;
 estado: 'disponible' | 'ocupado' | 'mantenimiento';
 capacidad: number;
 equipos: number;
}
```

Universidad Tecnológica del Perú

Pruebas antes de terminar

Comprueba que entendiste, no solo que se ve bien

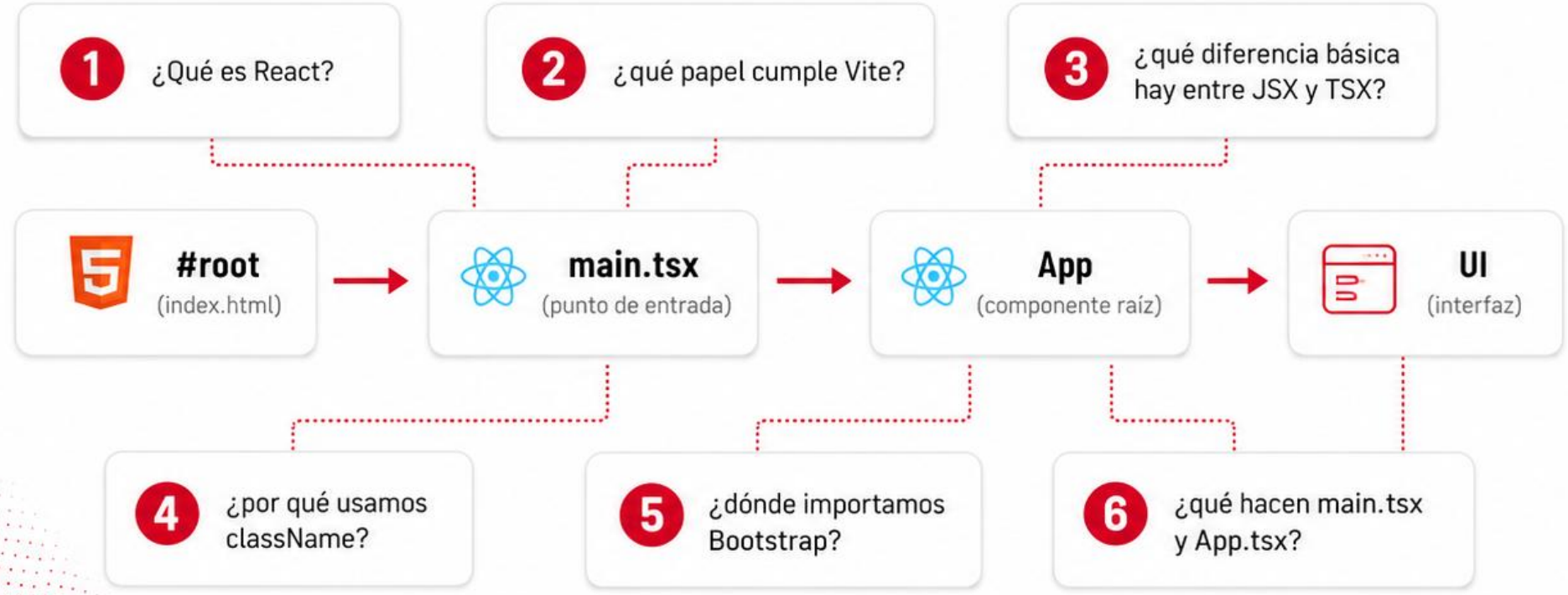
1. ¿`npm run dev` inicia sin error? ✓
2. ¿cada objeto cumple la interface? ✓
3. ¿la lista se genera con map()? ✓
4. ¿className se usa correctamente? ✓
5. ¿el contador de disponibles coincide? ✓
6. Cambia un laboratorio y verifica Fast Refresh. ✓
7. Finalmente ejecuta `npm run build.` ✓



Recuerda: no basta con que se vea bien.
Tu código debe funcionar, ser consistente y estar listo para producción.

Reconstruimos la sesión

Primero responde el grupo; después contrastamos



Metacognición y comprobación del logro

Qué aprendiste y qué todavía debes reforzar

Preguntas de reflexión

¿Qué fue más difícil: crear el proyecto, entender JSX/TSX o convertir datos en interfaz?

¿Qué error te enseñó más?

¿Qué estrategia usarías para diagnosticar un proyecto que no inicia?

Mini reto

Mini reto: si un archivo TypeScript contiene JSX, ¿qué extensión debe usar?



Respuesta esperada: **.tsx**

Próxima semana:
navegación, componentes y formularios en React.



¡MUCHAS GRACIAS POR SU ATENCIÓN!

Universidad Tecnológica del Perú